Results

Study 1: Feasibility of Predicting Keep/Remove Decisions

Motivation

We asked users to evaluate whether, if shown a social media post and its mirrored version (which preserves topic and intensity but flips the political stance), users would choose to keep both posts or remove both posts from a hypothetical social media platform. We then trained ML models to see if we could predict, given a post and its mirrored counterpart, whether participants would, on average, choose to keep or remove the pair.

Dataset

Prolific participants (n=1,321) reviewed pairs of original and mirrored posts (n=959 pairs), resulting in a final dataset of n=13,250 rows, with each row representing a human keep/remove decision for the pair of posts. In these "linked fate" trials, the users saw both the original post and its mirrored equivalent, randomly ordered, and were asked to make a keep/remove decision for the pair. In this dataset, 66.2% of post pairs were kept and 33.8% were removed. We create training labels for each post by taking the modal label across all raters (average n=13.8 labels per post).

Training an initial set of models with explainable text features

We first generated a set of explainable text features:

- Does the text involve intergroup discussion?
- Does the text have PRIME content?
- Does the text have a positive valence?
- Does the text have toxic content?
- What is the political stance of the text?
- What is the number of characters in the text?
- What is the average sentence length?
- What is the Flesch-Kincaid reading score?

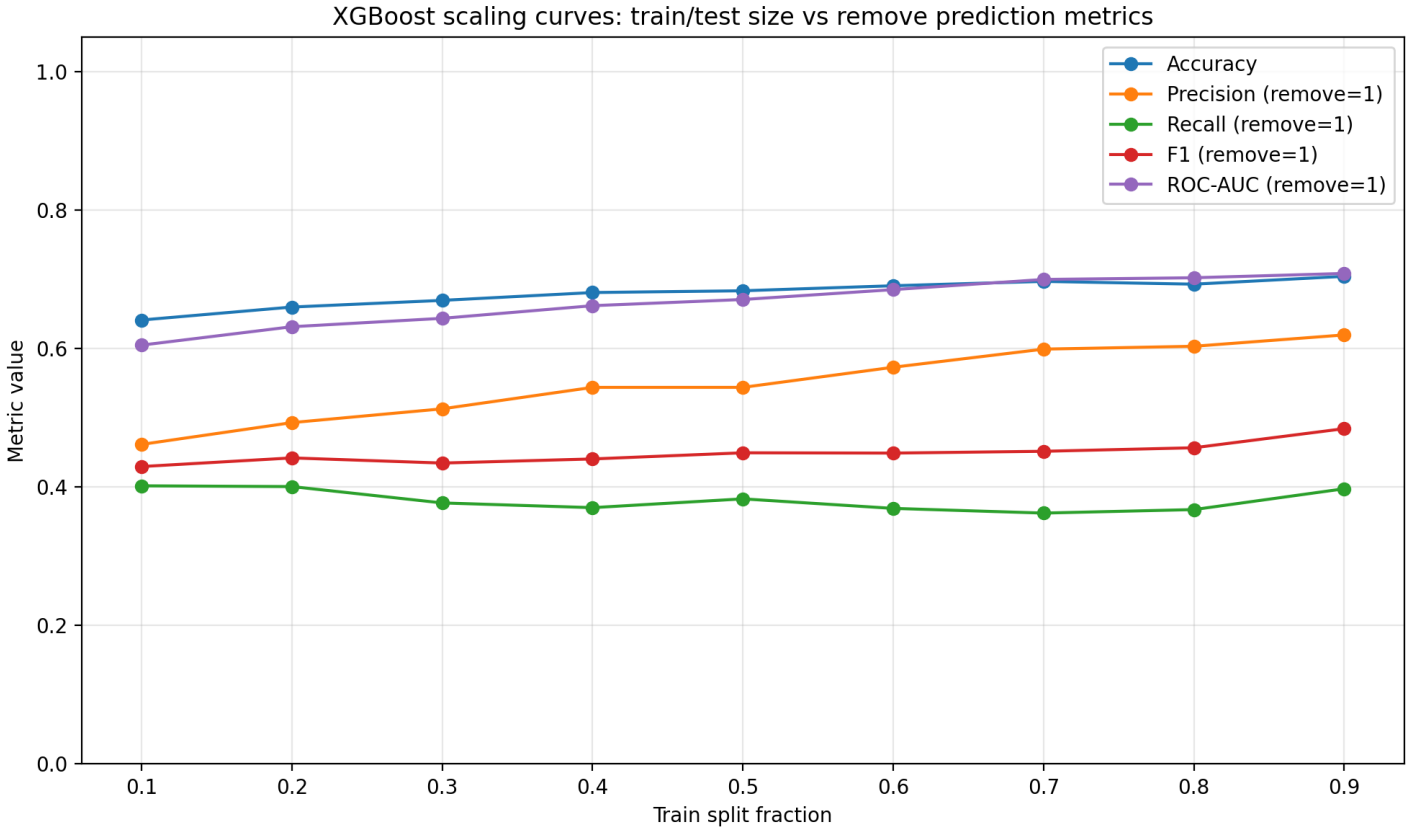
We then trained two models on these features: a logistic regression model and an XGBoost model. We report those results in a table

We report the model performance for predicting keep/remove decisions. Precision, recall, F1, and ROC-AUC are computed with the remove decision as the positive class ($y = 1$).

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (always predict keep)	Test	0.649	0.000	0.000	0.000	0.500
Logistic regression	Train	0.699	0.572	0.407	0.475	0.675
Logistic regression	Test	0.695	0.597	0.402	0.480	0.678
XGBoost	Train	0.737	0.667	0.430	0.523	0.777
XGBoost	Test	0.693	0.603	0.367	0.457	0.702

We notice a slight lift in the logistic regression and XGBoost models as compared to a naive baseline, driven by improvements in recall of remove decisions. We also observe likely overfitting on the XGBoost model given the sparsity of our n=959 dataset.

Scaling curves We also generated scaling curves by varying the proportion of the n=959 posts used in the training vs. test sets.



From our scaling curves, we don't see much improvement in recall and we only see slight gains in precision and overall accuracy. We have a small dataset size overall so it's unlikely that we have the sample size to learn sufficient signal.

Training models based on the text embeddings (Study 2)

Dataset (embedding models) We also trained additional predictive models based on text embeddings. We used Amazon Bedrock's Titan Text Embeddings (`amazon.titan-embed-text-v2:0`), with `d=256` dimensions. The embeddings were also L2-normalized.

We generated the following vectors:

- The original embedding vector, with shape `(256,)`.
- The mirrored post's embedding vector, with shape `(256, )`.
- The elementwise absolute difference between the original and mirrored post's vectors, with shape `(256, )`.
- The Hadamard elementwise product between the original and mirrored post embedding vectors, with shape `(256, )`.
- The scalar cosine similarity between the original and mirrored posts, with shape `(1,)`.
- One-hot vector of the original post's political stance (left vs. right), with shape `(2,)`.
- One-hot vector of the original post's toxicity category (low, middle, high), with shape `(3,)`.

We then stack these vectors into a single feature vector with shape `(1030,)`, creating a dataset with shape `(959, 1030)` for our posts.

Results (embedding models) We show the model performance for predicting keep/remove decisions using Bedrock text embeddings. Precision, recall, F1, and ROC-AUC are computed with the remove decision as the positive class ($y = 1$).

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (always predict keep)	Test	0.649	0.000	0.000	0.000	0.500
Logistic regression	Train	0.897	0.686	0.944	0.795	0.966
Logistic regression	Test	0.849	0.620	0.756	0.681	0.879
XGBoost	Train	1.000	1.000	1.000	1.000	1.000
XGBoost	Test	0.859	0.750	0.512	0.609	0.860

We observe that logistic regression generalizes better on the test set in terms of F1 and recall. XGBoost overfits more strongly, which we believe to be a symptom of the relatively small sample size. However, overall, training on the text embeddings themselves was much more performant than training on the hand-crafted text features.

We compare the results of the logistic regression and XGBoost models trained on the hand-crafted features versus the text embeddings in the following graph:

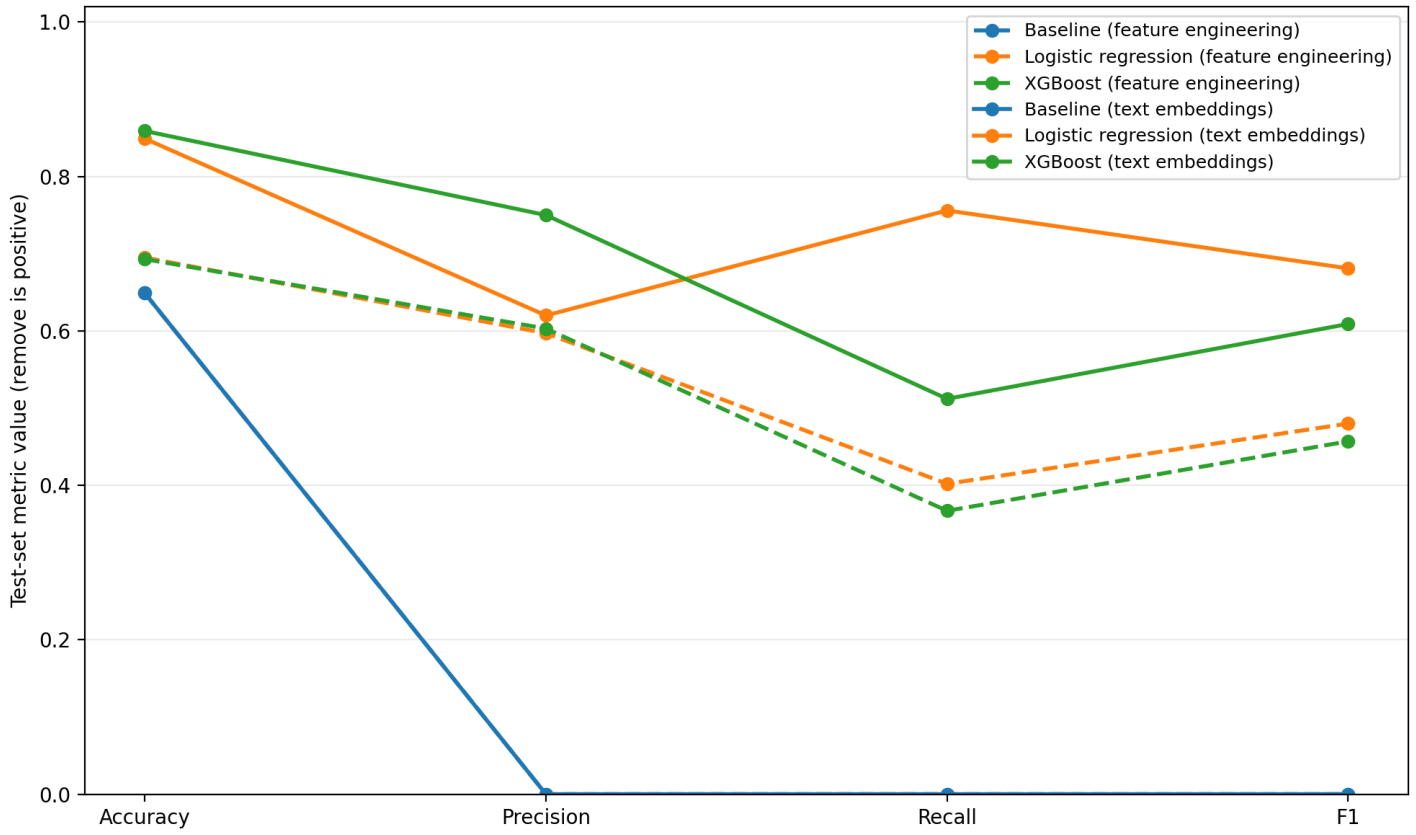


Figure 1: Test-set metric line graph

Study 2

In Study 2, we expanded on the work from Study 1.

Dataset

Prolific participants (n=1,176) reviewed pairs of original and mirrored posts (n=8,791 pairs), resulting in a final dataset of n=23,560 rows, with each row representing a human keep/remove decision for the pair of posts. In these "linked fate" trials, the users saw both the original post and its mirrored equivalent, randomly ordered, and were asked to make a keep/remove decision for the pair. In this dataset, 68% of post pairs were kept and 32% were removed. We create training labels for each post by taking the modal label across all raters (average n=2.68 labels per post).

Training models based on the text embeddings

We chose to focus on using text embeddings as features for our predictive models based on their vastly superior performance as compared to the hand-crafted features.

Training logistic regression and XGBoost models

Ablation 1: Concatenating the original post and mirrored post's embedding vectors We trained a series of logistic and XGBoost models. We first trained a series of models using feature vectors generated by concatenating the original post's embedding vector and the mirrored post's embedding vectors:

- The original post's embedding vector, with shape `(256,)`.

- The mirrored post's embedding vector, with shape `(256, )`.

We then stack these vectors into a single feature vector with shape `(512,)`, creating a dataset with shape `(8,791, 512)` for our posts.

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic regression	Train	0.719	0.548	0.689	0.611	0.777
Logistic regression	Test	0.671	0.490	0.623	0.548	0.716
XGBoost	Train	0.999	0.997	1.000	0.998	1.000
XGBoost	Test	0.720	0.583	0.435	0.498	0.718

Baseline results (original + mirrored post embeddings)

We first trained a series of models using feature vectors generated by concatenating the original post's embedding vector and the mirrored post's embedding vectors:

- The original post's embedding vector, with shape `(256,)`.
- The mirrored post's embedding vector, with shape `(256, )`.
- The cosine similarity of the two embeddings, as a float value with shape `(1,)`.

We then stack these vectors into a single feature vector with shape `(513,)`, creating a dataset with shape `(8,791, 513)` for our posts.

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic regression	Train	0.719	0.548	0.689	0.611	0.777
Logistic regression	Test	0.671	0.490	0.623	0.548	0.716
XGBoost	Train	0.999	0.997	1.000	0.998	1.000
XGBoost	Test	0.720	0.583	0.435	0.498	0.718

Ablations for training the logistic and XGBoost models

We also defined a series of ablations for the data used for training:

Name	Description (1 sentence)	Relevant train python files
`Original post + mirrored post embeddings`	Train on concatenated original and mirrored embeddings plus a cosine similarity feature.	`models/logistic_regression/train.py`, `models/xgboost/train.py`
`difference_embedding (orig_emb - mirror_emb)`	Train on the elementwise embedding difference (orig_emb - mirror_emb) only (no mirror embedding, no cosine).	`models/logistic_regression/train_differe` `models/xgboost/train_difference_embeddin`
`only_original_post_embedding`	Train on the original post embedding vector only (no mirror embedding, no cosine).	`models/logistic_regression/train_only_or` `models/xgboost/train_only_original_post_`

Ablation results

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic regression (Original post + mirrored post embeddings)	Train	0.719	0.548	0.689	0.611	0.777
Logistic regression (Original post + mirrored post embeddings)	Test	0.671	0.490	0.623	0.548	0.716
Logistic regression (difference embedding)	Train	0.620	0.433	0.612	0.507	0.663
Logistic regression (difference embedding)	Test	0.588	0.398	0.565	0.467	0.607
Logistic regression (only original)	Train	0.692	0.515	0.659	0.578	0.748
Logistic regression (only original)	Test	0.673	0.491	0.623	0.549	0.710
XGBoost (Original post + mirrored post embeddings)	Train	0.999	0.997	1.000	0.998	1.000
XGBoost (Original post + mirrored post embeddings)	Test	0.720	0.583	0.435	0.498	0.718
XGBoost (difference embedding)	Train	0.997	0.992	0.998	0.995	1.000
XGBoost (difference embedding)	Test	0.660	0.446	0.258	0.327	0.607
XGBoost (only original)	Train	0.993	0.983	0.996	0.990	1.000
XGBoost (only original)	Test	0.708	0.553	0.460	0.502	0.704

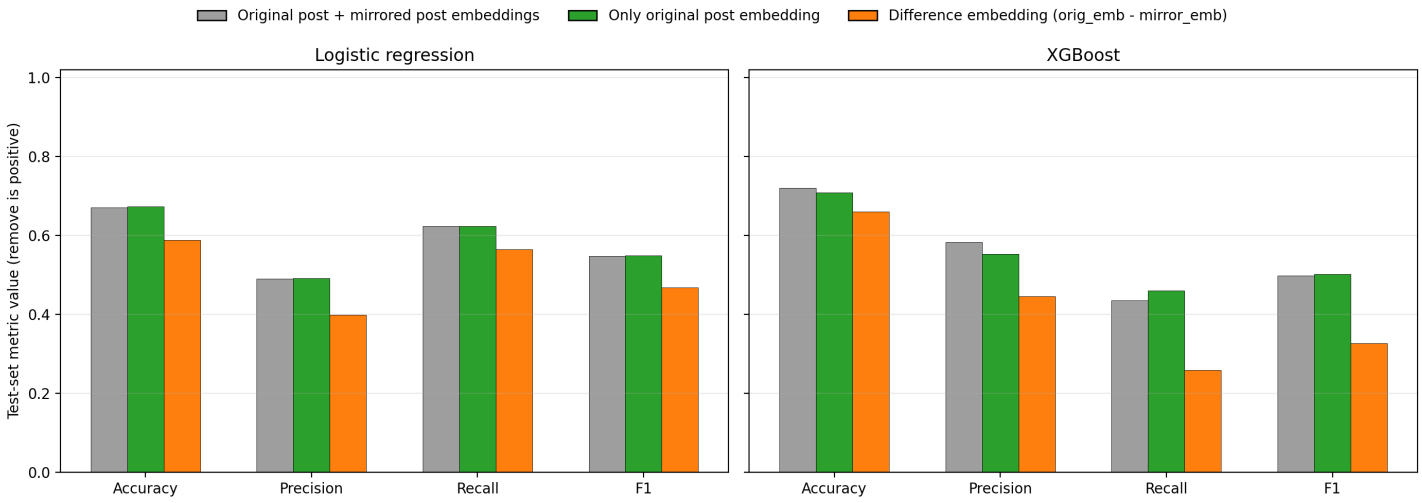


Figure 2: Ablation test-set metric bar graph

We observe that training a model on only the text embedding of the original post is as performant or more performant than the other ablations.

Even though adding the mirrored post's text embedding doesn't improve a model's predictive performance, we know that displaying the mirrored post in addition to the original post is significant. In other work, we show that there is a significant difference in the probability of removal based on whether a participant was exposed to our "linked fate" procedure of showing both the original post and its mirror.

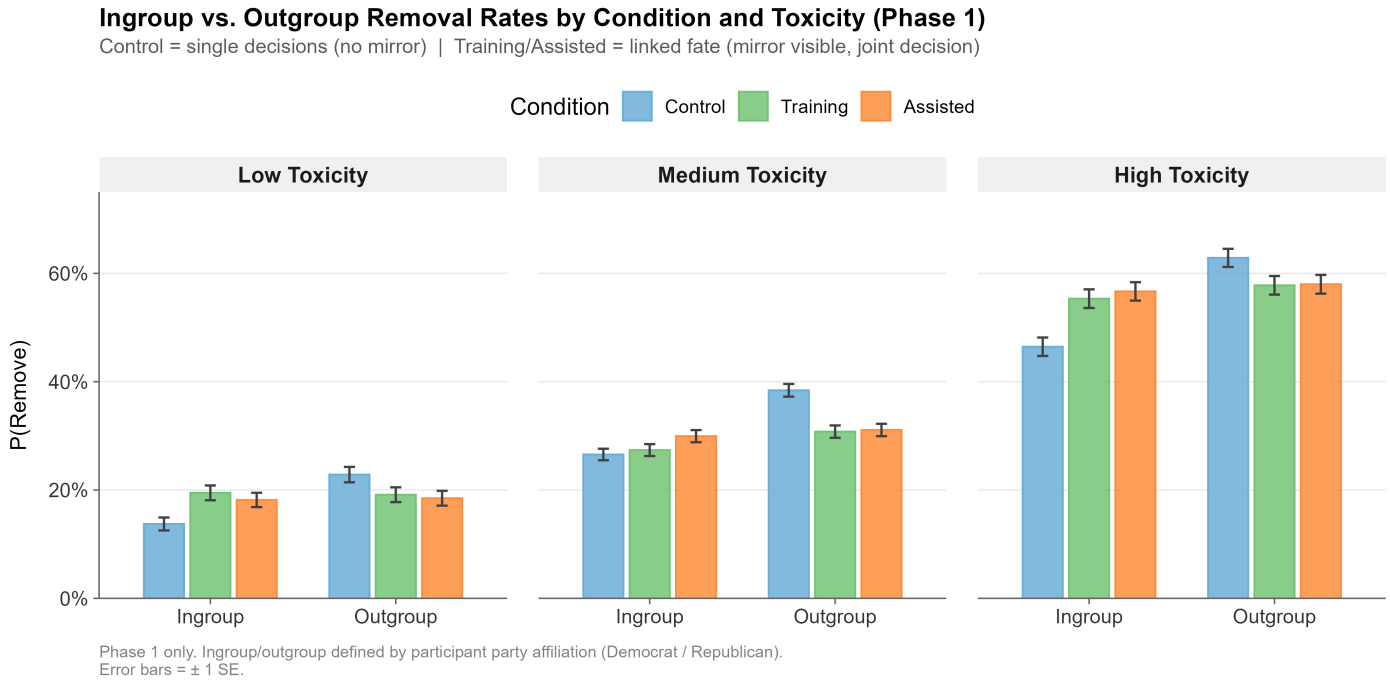


Figure 3: Removal rates by condition

We observe that the average cosine similarity between the original posts and their mirrored counterparts is 0.549. Most of the uniqueness of the content is driven by differences in the posts themselves, rather than between a post and its mirrored counterpart.

This implies that it is not the content of the mirrored text itself that affects the decision. Instead, the decision to remove posts is driven by (1) the content of the original post (of which the mirrored post is a similar ideological mirror) and (2) the mere presence of the mirrored post alongside the original post.

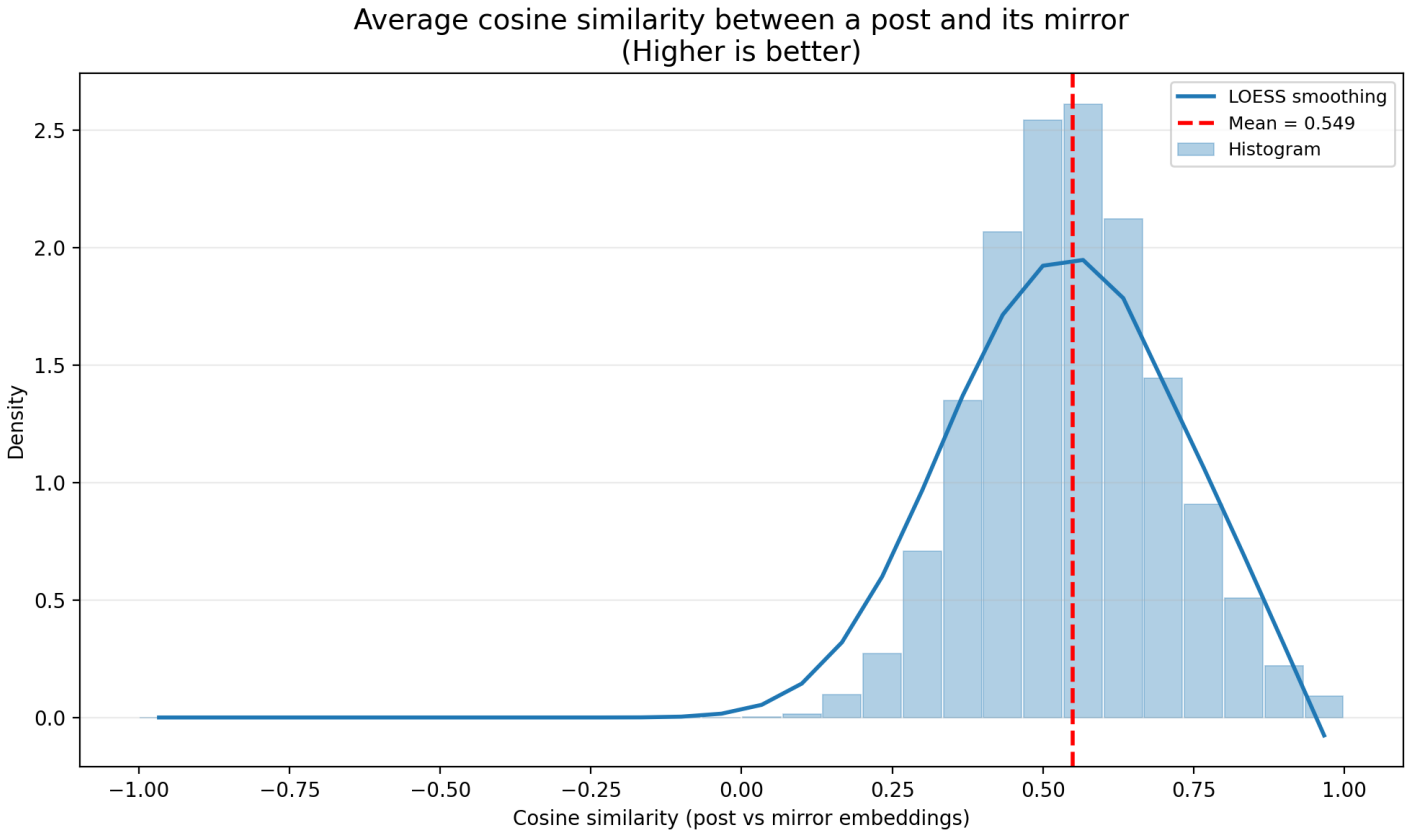


Figure 4: Average cosine similarity between a post and its mirror

Because we observe that there is no statistically significant difference in model performance between a model trained on just the original post's embeddings versus a model trained on both the original post and the mirrored post's embeddings, for parsimony reasons we chose to train subsequent models on just the text embeddings of the original post.

Exploring ingroup vs. outgroup content

(change the name of this, but basically the idea is how does training work depending on if the original post was ingroup or outgroup for that person?).

Fine-tuning a language model

(LoRA tuning?)

Scaling curves (Study 2)

(put scaling curves here)

Explainability

(explainability work)

Developing hand-crafted explainable features We compiled a taxonomy with the following features:

- (foo bar)

(Exploratory analyses of the explainable features and if we see any trends).

Training a model using the explainable features (we can train on both (1) just the top N explainable features and (2) text embeddings + top N explainable features).

Calibration

(calibration work)